Vulnerability Assessment

DVWA (Damn Vulnerable Web Application) — Lab Penetration Test Write-up

Author	Abinesh Sivakumar
Environment	Kali Linux (attacker) + DVWA on Apache/MySQL (target)
Isolation	VirtualBox / VMware host-only network
Scope	Self-hosted, intentionally vulnerable application — no external or production systems tested
Date	20 July 2026

All testing in this report was performed exclusively against a self-hosted, intentionally vulnerable lab application (DVWA) in an isolated host-only network, for educational purposes only.

---

1. Objective

To identify, exploit, and document common web application vulnerabilities in a controlled lab environment, and to practise the methodology used in real-world penetration testing: **recon** → **exploitation** → **proof-of-concept** → **remediation.**

2. Lab Setup

Element	Detail
Host	VirtualBox / VMware, host-only network (no internet-facing exposure)
Target	DVWA (security level set to Low, then Medium, to show escalating difficulty)
Attacker Machine	Kali Linux
Tools Used	Burp Suite / browser dev tools, sqlmap (optional), manual payloads

3. Findings

Finding 1 — SQL Injection

HIGH

Location DVWA → SQL Injection module

Vulnerability The `id` parameter is concatenated directly into a SQL query without sanitisation, allowing an attacker to alter query logic.

Steps to reproduce:

1. Navigate to the SQL Injection page and submit a benign ID (e.g. `1`) to confirm normal output.
2. Submit a payload such as `1' OR '1'='1` (or `1' UNION SELECT user, password FROM users --`) in the input field.
3. Observe that the application returns unintended data (e.g. all user records / credentials).

Impact An attacker could extract sensitive data (usernames, password hashes) from the underlying database, potentially leading to full account compromise.

Remediation:

- Use parameterised queries / prepared statements instead of string concatenation.
- Apply input validation and least-privilege database accounts.
- Enable a Web Application Firewall (WAF) as a defence-in-depth measure.

Finding 2 — Cross-Site Scripting (XSS)

MEDIUM

Location DVWA → XSS (Reflected or Stored) module

Vulnerability User input is rendered back to the page without output encoding, allowing injection of arbitrary HTML/JavaScript.

Steps to reproduce:

1. Submit a payload such as `<script>alert('XSS-PoC-Abinesh')</script>` into the vulnerable input field.
2. Observe that the script executes in the browser when the page renders the input.

Impact In a real application, this could be used to steal session cookies, perform actions on behalf of a victim user, or redirect users to malicious sites.

Remediation:

- Encode all user-supplied output (HTML-entity encode by context).
- Apply a Content Security Policy (CSP).
- Validate and sanitise input server-side, never trust client-side validation alone.

Finding 3 — OS Command Injection CRITICAL

Location DVWA → Command Injection module

Vulnerability The application takes a user-supplied IP address and passes it directly into a system shell command (a `ping` call) without sanitisation, allowing shell metacharacters to append arbitrary operating system commands.

Steps to reproduce:

1. Navigate to the Command Injection page and submit a valid IP address (e.g. `127.0.0.1`) to confirm the expected ping output.
2. Submit a chained payload such as `127.0.0.1 && whoami` (or `127.0.0.1; id`) in the input field.
3. Observe that the output of the injected command (e.g. the current user) is returned alongside the ping results, confirming arbitrary command execution.

Impact An attacker could execute arbitrary operating system commands with the privileges of the web server process — enabling file read/write, credential harvesting, installation of a reverse shell, or use of the compromised host as a pivot point into the wider network.

Remediation:

- Avoid invoking OS shell commands from application code; use safe, built-in language libraries/APIs (e.g. native ping/network libraries) instead of shelling out.
- Strictly validate and whitelist input against an expected format (e.g. a valid IPv4/IPv6 regex).
- Run the web application process under a least-privilege service account and sandbox/containerise it.

6. Summary Table

#	Vulnerability	Severity	Status
1	SQL Injection	High	Reproduced & documented
2	Cross-Site Scripting (XSS)	Medium	Reproduced & documented
3	OS Command Injection	Critical	Reproduced & documented

7. Lessons Learned

This exercise showed how directly vulnerabilities trace back to a handful of insecure coding habits — trusting user input, building queries and shell commands through string concatenation, and rendering output without encoding it. Working through recon, exploitation, proof-of-concept, and remediation for each finding reinforced that identifying a bug is only half the job; explaining its real-world impact and a concrete fix is what makes the write-up useful to a development team. It also clarified the kind of offensive security role I want to grow into: one where I'm not just breaking things in a lab, but building the habit of thinking like a defender at the same time.

All testing was performed exclusively against a self-hosted, intentionally vulnerable lab application (DVWA) for educational purposes.

— End of Report —